

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING, HCMUT



LOGIC DESIGN WITH HDL (LAB)

ASSIGNMENT

Instructor: **Pham Kieu Nhat Anh**

Group: 4 Class: CC01

Name	Student ID
Nguyễn Đình Thúc Nguyên	2550318
Bùi Trần Quyền	2353033
Vũ Thiện Nhân	2452888
Hà Hoàng Ân	2452106
Trần Đăng Huy	2510316

Ho Chi Minh City, June 2026

Table of Contents

Task Allocation and Contributions	3
I Introduction	4
I.1 Overview of the Project	4
I.2 Motivation and Scope	4
I.3 Features of the Final Product	4
II Theoretical Overview	5
II.1 RISC Architecture	5
II.2 Instruction Set Architecture	5
II.3 Processor Data Path	6
II.4 Functional Block Descriptions	7
II.4.1 Program Counter (PC)	7
II.4.2 Address Multiplexer (Address MUX)	7
II.4.3 Memory	7
II.4.4 Instruction Register (IR)	7
II.4.5 Accumulator Register	7
II.4.6 Arithmetic Logic Unit (ALU)	7
II.4.7 Controller	8
II.5 Timing and Synchronisation	8
III Design	8
III.1 Our RISC Architecture	8
III.2 Functional Block Descriptions	9
III.2.1 Program Counter (PC)	9
III.2.2 Address Multiplexer	9
III.2.3 Instruction Register (IR)	10
III.2.4 Accumulator Register (AC)	10
III.2.5 Memory	10
III.2.6 Arithmetic Logic Unit (ALU)	10
III.2.7 Controller (FSM)	11
III.3 System Workflow	11
IV Implementation	12
IV.1 Bidirectional Data Bus	12
IV.2 Controller FSM	12
IV.3 ALU and Datapath	13
V Simulation and Result	13
V.1 Unit Testing (Module-Level Verification)	13
V.1.1 Arithmetic Logic Unit (ALU)	14
V.1.2 Memory and Tri-State Bus	14
V.1.3 Controller (Finite State Machine)	14
V.1.4 PC, IR, AC, MUX	15

V.2	Integration Testing (System-Level Verification)	16
V.2.1	Test Case 1: Arithmetic, Logic, and Store Sequence (<code>input1.txt</code>) . .	16
V.2.2	Test Case 2: Endless Execution Without HLT (<code>input2.txt</code>)	17
V.3	Performance and Synthesis Evaluation	17
V.3.1	Instruction Latency	17
V.3.2	Throughput	18
V.3.3	Program Execution Time	18
VI	Conclusions	19

Task Allocation and Contributions

Table 1: Work allocation and contributions by team members

Name	Student ID	Responsibilities
Nguyễn Đình Thúc Nguyễn	2550318	Information summarisation and report writing.
Bùi Trần Quyền	2353033	PC and Controller (FSM) design.
Vũ Thiện Nhân	2452888	ALU and Memory design.
Hà Hoàng Ân	2452106	Top-level block, system testbench, and bonus feature design.
Trần Đăng Huy	2510316	Address MUX, IR, and AC design.

I Introduction

I.1 Overview of the Project

This project focuses on the design and implementation of a simple 8-bit Reduced Instruction Set Computer (RISC) processor using Verilog HDL. The processor follows a von Neumann architecture, where instructions and data share the same 32-location memory space. It supports eight instructions—HLT, SKZ, ADD, AND, XOR, LDA, STO, and JMP—encoded by a 3-bit opcode and a 5-bit operand.

The system is organized into seven main functional blocks: Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, Arithmetic Logic Unit, and an eight-state finite state machine Controller. Operating with a clock-driven datapath and an active-high synchronous reset, the processor performs instruction fetch, decode, operand access, execution, and write-back before halting when a HLT instruction is encountered. Through this modular design, the project demonstrates key RTL design concepts such as FSM control, shared bus organization, and processor datapath integration.

I.2 Motivation and Scope

Understanding processor design at the register-transfer level (RTL) is a foundational competency in computer engineering. This project bridges the gap between abstract digital logic and tangible computing systems by requiring students to reason about timing, control signals, data paths, and instruction sequencing simultaneously. By designing each functional unit independently and verifying it through simulation, the team gains practical experience in modular hardware design—a discipline central to modern VLSI and embedded systems development.

The scope of this project covers the full design cycle of the processor: architectural specification, RTL-level module implementation in Verilog HDL, functional verification through testbenches, and integration into a top-level system. Hardware synthesis and physical implementation are outside the scope of this report.

I.3 Features of the Final Product

The completed processor design embodies the following features:

- **Instruction Set Architecture (ISA):** An eight-instruction ISA with 3-bit opcodes supporting arithmetic (ADD), logical (AND, XOR), memory access (LDA, STO), flow control (JMP, SKZ), and program termination (HLT) operations.
- **Modular RTL Design:** The processor is decomposed into seven independently verifiable functional blocks: Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, Arithmetic Logic Unit (ALU), and Controller.
- **Synchronous Operation:** All sequential elements are clocked on the positive edge of a common clock signal, ensuring predictable and race-free behaviour.

- **Eight-State Fetch-Decode-Execute Cycle:** The controller orchestrates an eight-state finite state machine (FSM) covering instruction address generation, instruction fetch, instruction load, idle, operand address generation, operand fetch, ALU operation, and result store.
- **Unified Memory Architecture:** A single bidirectional-port memory block stores both instructions and data, with explicit read/write control signals preventing simultaneous access conflicts.
- **Per-Module Testbenches:** Each functional block is accompanied by a dedicated Verilog testbench exercising its boundary conditions and nominal operating cases.

II Theoretical Overview

II.1 RISC Architecture

A Reduced Instruction Set Computer (RISC) is a processor design philosophy characterised by a small, highly optimised set of instructions, each of which executes in a fixed and predictable number of clock cycles. In contrast to Complex Instruction Set Computer (CISC) architectures, RISC designs favour instruction-set simplicity and rely on the compiler or programmer to synthesise complex operations from primitive ones. This simplicity yields several engineering advantages: reduced decoding logic, more regular pipeline stages, and lower control complexity.

The processor designed in this project adheres to core RISC principles. Instructions are fixed-width 8-bit words composed of a 3-bit opcode and a 5-bit operand field. All arithmetic and logical operations occur between the dedicated Accumulator register and a memory-resident operand; there are no general-purpose register files. This accumulator-based model is representative of early RISC and microcontroller architectures and serves as a clear pedagogical vehicle for understanding data-path and control-path design.

II.2 Instruction Set Architecture

The processor supports eight instructions, each uniquely identified by a 3-bit opcode. The full instruction set is summarised in Table 2.

Table 2: Instruction Set Architecture of the RISC Processor

Mnemonic	Opcode	Description
HLT	000	Halts processor execution.
SKZ	001	Skips the next instruction if the current ALU output is zero; otherwise, execution continues sequentially.
ADD	010	Adds the value at the specified memory address to the Accumulator and stores the result back in the Accumulator.
AND	011	Performs a bitwise AND between the Accumulator and the value at the specified memory address, then stores the result in the Accumulator.
XOR	100	Performs a bitwise XOR between the Accumulator and the value at the specified memory address, then stores the result in the Accumulator.
LDA	101	Loads the value at the specified memory address into the Accumulator, replacing its current contents.
STO	110	Stores the current value of the Accumulator into the specified memory address.
JMP	111	Unconditional jump: loads the 5-bit operand into the Program Counter, transferring control to that address.

II.3 Processor Data Path

The data path defines how information flows between the functional blocks of the processor during each stage of execution. At a high level, the processor follows a *fetch-decode-execute* cycle:

1. **Instruction Fetch:** The Program Counter (PC) supplies the current instruction address to the Address MUX. The MUX forwards this address to Memory, which outputs the 8-bit instruction word. The Instruction Register (IR) captures and holds this word.
2. **Instruction Decode:** The Controller interprets the upper 3 bits of the IR (the opcode) and generates the appropriate control signals for the subsequent execution stage.
3. **Operand Fetch (if required):** For instructions that reference a memory operand (ADD, AND, XOR, LDA, STO), the lower 5 bits of the IR (the address field) are routed through the Address MUX to Memory, which returns the operand.

4. **Execute:** The ALU performs the operation dictated by the opcode on the Accumulator value and, where applicable, the memory operand. The result is written back to the Accumulator or to Memory as required.

II.4 Functional Block Descriptions

II.4.1 Program Counter (PC)

The Program Counter is a 5-bit synchronous register that holds the address of the instruction currently being fetched. It increments by one on each fetch cycle unless overridden by a **JMP** instruction, in which case a 5-bit target address is loaded directly into the counter. An active-high, synchronous reset returns the PC to address 0x00.

II.4.2 Address Multiplexer (Address MUX)

The Address MUX is a parameterised 2-to-1 multiplexer with a default data width of 5 bits. Its select signal, driven by the Controller, determines whether the PC output (instruction address) or the IR address field (operand address) is presented to the Memory unit. This single component allows the memory bus to be shared between the instruction fetch and operand fetch phases without requiring separate buses.

II.4.3 Memory

The Memory is a synchronous single-port RAM with a 5-bit address space (32 locations) and an 8-bit data width. A unified bidirectional data port allows the same port to serve both read and write operations; simultaneous reads and writes are not permitted. Separate 1-bit read and write enable signals gate each access, and all operations are registered on the positive clock edge.

II.4.4 Instruction Register (IR)

The Instruction Register is an 8-bit synchronous register that captures the instruction word presented on the memory data bus during the instruction fetch stage. It is loaded under the control of the **ld_ir** signal asserted by the Controller. Once loaded, the IR retains the instruction word for the duration of the decode and execute stages, providing a stable source of opcode and address-field information.

II.4.5 Accumulator Register

The Accumulator is an 8-bit synchronous register that holds the primary operand and result of all arithmetic and logical operations. It is updated on the positive clock edge when the **ld_ac** control signal is asserted. Its output feeds one of the two inputs (**inA**) of the ALU.

II.4.6 Arithmetic Logic Unit (ALU)

The ALU is a combinational block that accepts two 8-bit operands (**inA** from the Accumulator and **inB** from Memory) and produces an 8-bit result along with a 1-bit **is_zero**

flag. The `is_zero` signal is asynchronous and indicates whether `inA` is equal to zero; it is used by the `SKZ` instruction. The operation performed is selected by the 3-bit opcode from the IR, following the encoding in Table 2.

II.4.7 Controller

The Controller is the central finite state machine of the processor. It is a Moore-type synchronous FSM with eight states, traversed sequentially over eight consecutive clock cycles: `INST_ADDR`, `INST_FETCH`, `INST_LOAD`, `IDLE`, `OP_ADDR`, `OP_FETCH`, `ALU_OP`, and `STORE`. On reset, the FSM returns to the `INST_ADDR` state. At each state, the Controller asserts a combination of nine output control signals—`sel`, `rd`, `ld_ir`, `halt`, `inc_pc`, `ld_ac`, `ld_pc`, `wr`, and `data_e`—whose values depend on both the current state and the opcode from the IR, as specified in the course assignment.

II.5 Timing and Synchronisation

All sequential elements in the processor (PC, IR, Accumulator, Memory, Controller) are clocked on the positive edge of a single global clock signal. This fully synchronous design eliminates metastability risks arising from asynchronous interactions between blocks. The `is_zero` flag of the ALU is the sole asynchronous signal; it reflects the combinational output of the ALU instantaneously and is sampled by the Controller at the appropriate clock edge, ensuring correct `SKZ` behaviour without introducing timing hazards under normal operating conditions.

III Design

III.1 Our RISC Architecture

The RISC processor is constructed using a modular, block-level design approach. It integrates seven primary functional units that interact through a centralised Controller and a shared, bidirectional 8-bit data bus. The architecture employs a unified memory model in which both instructions and data are fetched from the same memory space using a multiplexed address bus.

Figure 1 illustrates the top-level block diagram of the processor, highlighting the data path and control-signal routing from the FSM Controller.

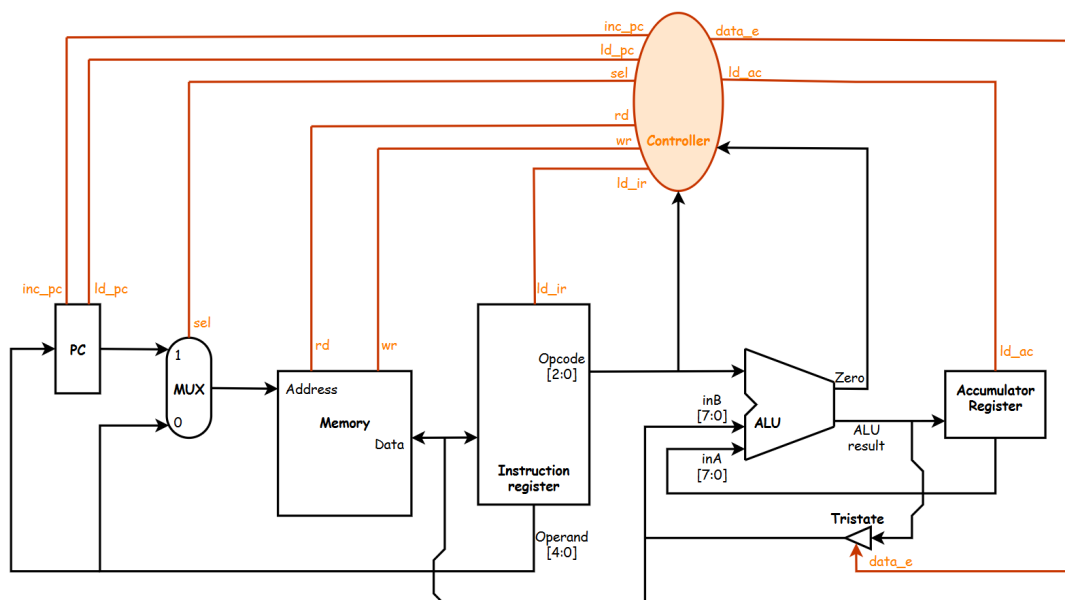


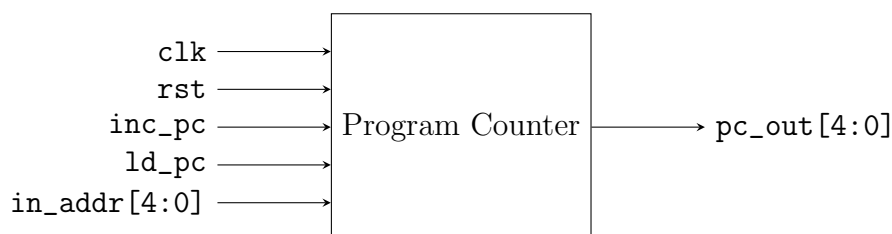
Figure 1: Top-Level Architecture of the RISC Processor

III.2 Functional Block Descriptions

The processor is divided into seven structural modules. Each module is designed to execute a specific hardware function synchronously on the positive edge of the system clock, except for certain asynchronous combinational read operations.

III.2.1 Program Counter (PC)

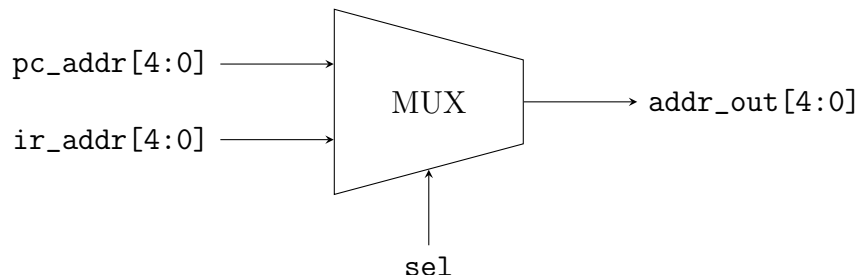
The Program Counter manages the 5-bit address of the instruction sequence. It is a synchronous register equipped with a reset mechanism. Under normal operation, asserting `inc_pc` increments the address by one. For branching instructions (e.g., `JMP`), asserting `ld_pc` allows the PC to overwrite its current state with the 5-bit `in_addr` provided by the Instruction Register.



III.2.2 Address Multiplexer

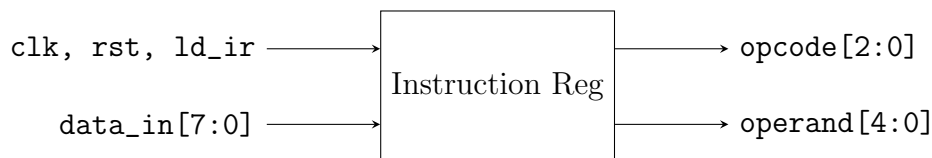
The Address Multiplexer routes the appropriate 5-bit address to the Memory module. Controlled by the `sel` signal from the FSM, it acts as a combinational switch. When `sel` = 1, it selects `pc_addr` during the Instruction Fetch phase. When `sel` = 0, it selects `ir_addr`,

which is extracted from the operand field of the instruction during the Operand Fetch and Store phases.



III.2.3 Instruction Register (IR)

The Instruction Register acts as the decoding boundary. Upon receiving the `ld_ir` signal, it latches the 8-bit word from the data bus. It statically partitions this word into two outputs: the upper 3 bits form `opcode[2:0]`, which is routed to the Controller and ALU, while the lower 5 bits constitute `operand[4:0]`, which is routed to the Address MUX and PC as either a memory address or jump target.



III.2.4 Accumulator Register (AC)

The Accumulator is the primary 8-bit working register of the RISC processor. It synchronously captures the 8-bit output from the ALU (`alu_out`) when the `ld_ac` flag is asserted by the Controller. The data stored in the AC (`ac_out`) is continuously fed back into the ALU as operand `inA`.

III.2.5 Memory

The Memory block is a 32-byte (32×8 -bit) array that stores both instructions and user data. It uses a unified bidirectional `data` bus controlled by a tri-state buffer mechanism.

- **Read Path:** Asynchronous and combinational. The memory drives the bus only when `rd = 1` and `wr = 0`.
- **Write Path:** Synchronous. The memory latches data from the bus on the positive clock edge only when `wr = 1` and `rd = 0`.

III.2.6 Arithmetic Logic Unit (ALU)

The ALU is a purely combinational arithmetic and logic engine. It evaluates the 3-bit `opcode` to determine the operation performed between `inA` (Accumulator) and `inB` (Data

Bus/Memory). In addition to generating the 8-bit `alu_out`, it dynamically asserts an `is_zero` condition when the value in the Accumulator (`inA`) is zero. This flag is critical for executing the `SKZ` instruction.

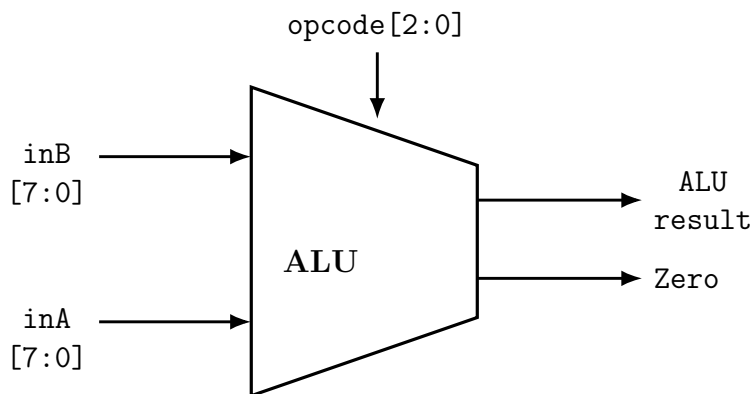


Figure 2: ALU block diagram.

III.2.7 Controller (FSM)

The Controller serves as the system's control unit and is designed as an eight-state Moore finite state machine. It receives the current `opcode` and the ALU's `zero` flag, advancing through the operation phases on each clock cycle. Depending on its state, it combinationaly drives the nine control lines that dictate multiplexer routing, register latching, memory read/write access, and Program Counter increments.

III.3 System Workflow

The operation of the processor revolves around an eight-state instruction cycle driven by the FSM. Regardless of the instruction's complexity, the machine sequentially cycles through these states to fetch, decode, and execute instructions.

The cycle breaks down as follows:

1. **INST_ADDR:** The multiplexer selects the PC address.
2. **INST_FETCH:** The memory read enable (`rd`) is asserted to retrieve the instruction.
3. **INST_LOAD:** The Instruction Register is enabled (`ld_ir`) to save the instruction word.
4. **IDLE:** A transitional padding state for timing synchronisation.
5. **OP_ADDR:** The PC increments (`inc_pc`). If the instruction is `HLT`, the FSM remains in this state permanently. Otherwise, it proceeds.
6. **OP_FETCH:** For memory-dependent instructions (e.g., `ADD`, `LDA`), the memory read line is enabled for the operand address.

7. **ALU_OP:** Memory operands are read. **SKZ** resolves conditional skips by manipulating the PC. **JMP** triggers `ld_pc`. For **STO**, `data_e` is asserted to place Accumulator data on the bus.
8. **STORE:** In this final step, ALU results are loaded back into the Accumulator (`ld_ac`), or Memory asserts write enable (`wr`) to capture the data bus for **STO**. The FSM then loops back to **INST_ADDR**.

IV Implementation

This project is written in Verilog HDL using a Register-Transfer Level (RTL) design style, separating synchronous sequential elements from combinational datapath and control logic.

IV.1 Bidirectional Data Bus

A critical aspect of the implementation is the 8-bit bidirectional data bus connecting the Memory, Instruction Register (IR), and Arithmetic Logic Unit (ALU). To prevent short circuits and data contention, tri-state buffers were modelled using the high-impedance state (8'bz).

Bus access is strictly gated by the Controller's signals:

- **Top-Level Bus Driver (Write Cycle):** The ALU or Accumulator writes to the bus only when the write enable (`wr`) and data enable (`data_e`) signals are asserted.

```
1 // From risc_processor_25.v
2 assign data_bus = (wr && data_e) ? alu_out : 8'bz;
```

- **Memory Bus Driver (Read Cycle):** The memory module drives the bus only during a read cycle (`rd`) when writing is explicitly disabled (`!wr`).

```
1 // From memory.v
2 assign data = (rd && !wr) ? mem[addr] : 8'bz;
```

IV.2 Controller FSM

The Controller is realised as a three-block Moore finite state machine.

- **State Encoding:** The eight states are encoded as 3-bit parameters, ranging from **INST_ADDR** (3'b000) to **STORE** (3'b111).
- **State Register:** A synchronous `always @(posedge clk)` block handles the sequential logic, updating the `current_state` or applying the synchronous reset.

```

1 // From controller.v
2 always @(posedge clk) begin
3     if (rst) begin
4         current_state <= INST_ADDR;
5     end else begin
6         current_state <= next_state;
7     end
8 end

```

- **Next-State and Output Logic:** Combinational always @(*) blocks resolve the next_state and control signals. For instance, the transition from OP_ADDR evaluates the HLT opcode to determine whether the processor should halt or fetch an operand.

IV.3 ALU and Datapath

The ALU is implemented purely combinationally to ensure that results are available within the same clock cycle in which they are requested.

- **Zero Flag:** The is_zero flag is continuously assigned based on the Accumulator's output (inA), enabling immediate evaluation for the SKZ instruction.

```

1 // From alu.v
2 assign is_zero = (inA == 8'b0) ? 1'b1 : 1'b0;

```

- **Arithmetic and Logic Operations:** A case statement decodes the opcode to execute data operations such as addition (ADD), bitwise AND (AND), and XOR (XOR).

```

1 // From alu.v
2 case (opcode)
3     ADD: alu_out = inA + inB;
4     AND: alu_out = inA & inB;
5     XOR: alu_out = inA ^ inB;
6     // ...

```

V Simulation and Result

Verification of the RISC processor was conducted using a bottom-up approach within the Cadence/Vivado simulation environments. Testing was divided into two distinct phases: unit testing of individual modules to verify specific hardware behaviours, and system-level integration testing to ensure that the processor accurately executes full programs.

V.1 Unit Testing (Module-Level Verification)

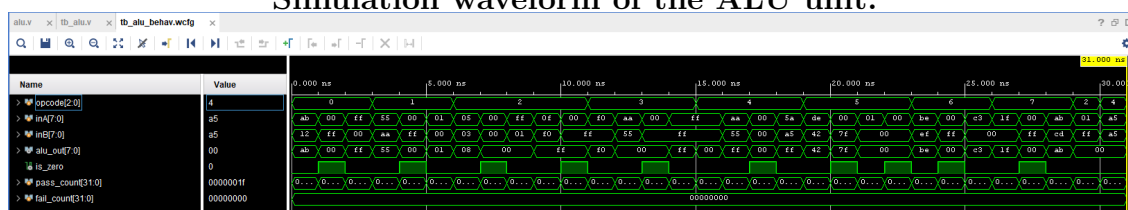
Each functional block was subjected to an isolated testbench to verify its logic, synchronous behaviour, and boundary conditions.

V.1.1 Arithmetic Logic Unit (ALU)

The ALU testbench (`tb_alu.v`) uses an automated self-checking mechanism with pass/fail counters. A total of 31 test cases were executed.

Analysis: The waveform confirms that the ALU output changes immediately according to the selected `opcode`, demonstrating the intended combinational behaviour. A notable point is that the `is_zero` flag follows the Accumulator input `inA`, not the computed `alu_out`. Therefore, even when an arithmetic result becomes zero, the zero flag is asserted only when `inA` itself is zero, which matches the intended behaviour for the SKZ instruction.

Simulation waveform of the ALU unit.

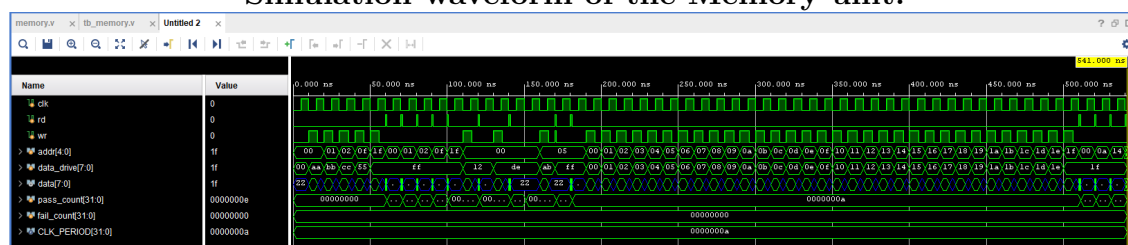


V.1.2 Memory and Tri-State Bus

The Memory testbench (`tb_memory.v`) evaluates the single-port RAM and its bidirectional bus interface across five distinct blocks.

Analysis: The waveform highlights the memory bus behaviour under read, write, idle, and conflict conditions. During write cycles, data is captured into the selected address only when `wr` is active and `rd` is inactive. During read cycles, the memory drives the bus only when `rd=1` and `wr=0`. The special high-impedance states are important because they show that the bidirectional bus is protected from unintended contention when no valid driver should be active.

Simulation waveform of the Memory unit.



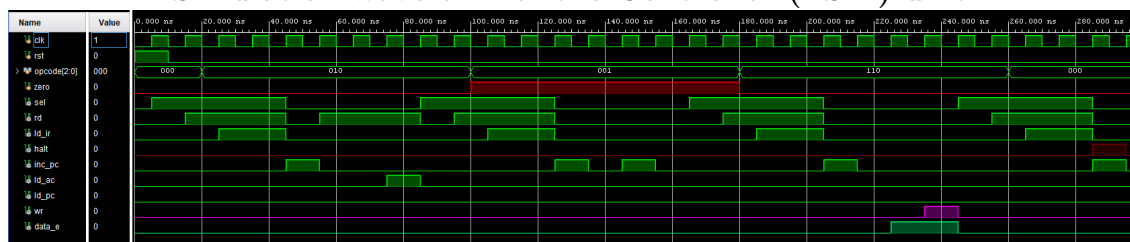
V.1.3 Controller (Finite State Machine)

The Controller testbench (`tb_controller.v`) simulates five major instruction workflows to verify the FSM transitions and control-signal assertions over the eight-cycle instruction execution.

Analysis: The waveform shows the Controller moving through the eight execution states in order while asserting the corresponding datapath control signals. The most important feature is the conditional behaviour of branch-related instructions. When SKZ is combined with an asserted zero flag, the controller produces the additional Program Counter increment

needed to skip the next instruction. For HLT, the waveform shows the halt condition holding the FSM instead of continuing the normal instruction cycle.

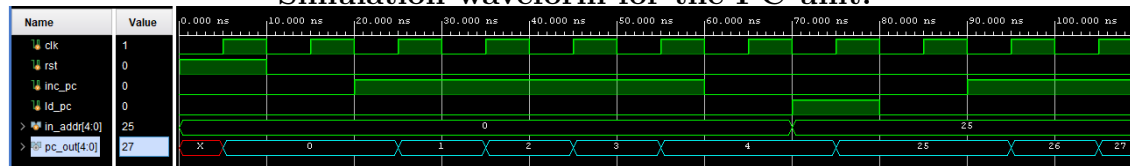
Simulation waveform of the Controller (FSM) unit.



V.1.4 PC, IR, AC, MUX

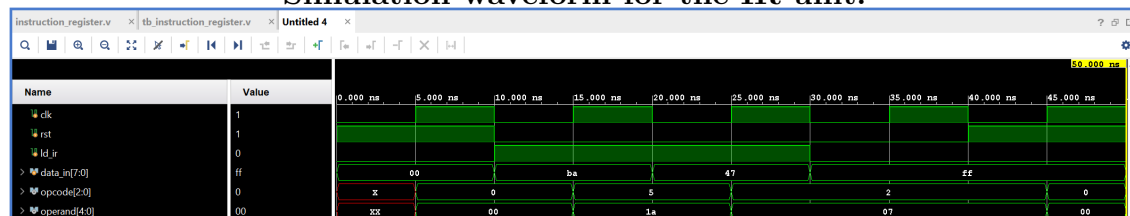
The remaining module waveforms demonstrate the smaller datapath elements that support instruction sequencing, decoding, storage, and address selection.

Simulation waveform for the PC unit.

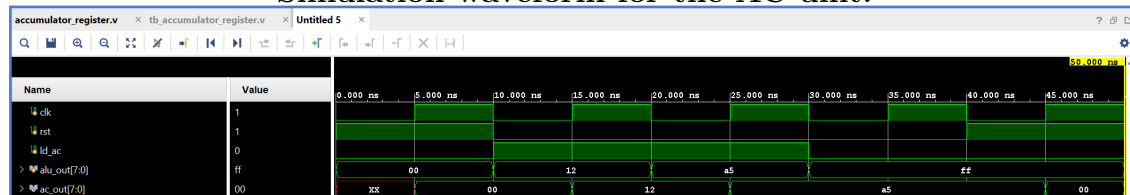


PC waveform analysis: The Program Counter waveform is notable for showing two distinct behaviours: sequential incrementing under `inc_pc` and direct address loading under `ld_pc`. This confirms that the PC can support both normal instruction flow and jump-style control flow.

Simulation waveform for the IR unit.

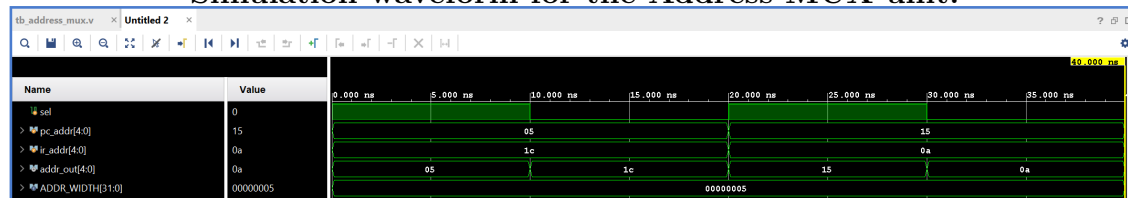


Simulation waveform for the AC unit.



IR and AC waveform analysis: The IR waveform is significant because the 8-bit instruction word is separated into a 3-bit opcode and a 5-bit operand only after the load signal is asserted, keeping the decoded fields stable between instruction loads. The AC waveform shows that accumulator updates occur only on enabled clock edges, which prevents unintended datapath changes while other stages of the processor are active.

Simulation waveform for the Address MUX unit.



Address MUX waveform analysis: The Address MUX waveform is notable for its immediate combinational response to the `sel` signal. When `sel` changes, the output switches between the Program Counter address and the Instruction Register operand address, confirming correct address routing for both instruction fetch and operand access.

V.2 Integration Testing (System-Level Verification)

System integration testing was performed using the top-level testbench (`tb_risc_processor.v`), which instantiated the entire processor and loaded machine-code programs into memory using the `$readmemb` system task. A custom monitor block was implemented to log the Program Counter, decoded instruction, and Accumulator value at every fetch cycle.

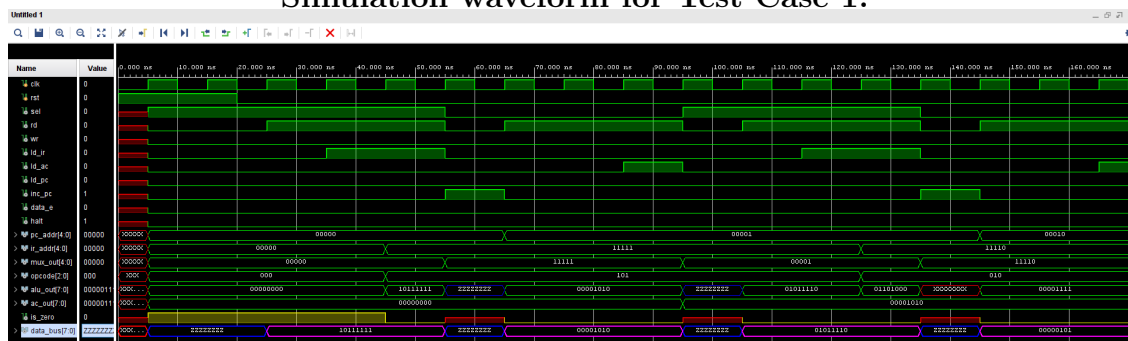
Four distinct programs were executed to evaluate the processor’s architectural reliability:

V.2.1 Test Case 1: Arithmetic, Logic, and Store Sequence (input1.txt)

This test verifies a straight-line program that combines memory loading, arithmetic addition, bitwise logic, memory storage, and final halt behaviour.

The program begins with **LDA 31**, loading the data value 10 from memory address 31 into the Accumulator. It then executes **ADD 30**, adding the value 5 and producing 15. Next, **AND 29** applies a bitwise AND with the value 7, reducing the Accumulator result to 7. The instruction **STO 28** stores this result into memory address 28, after which **LDA 28** reloads the stored value and **STO 27** writes it to another memory location before the program reaches **HLT**.

Simulation waveform for Test Case 1.



The waveform confirms the expected instruction sequence through the Program Counter, opcode, address, data bus, and Accumulator signals. During the first instruction, the Accumulator is loaded with 00001010; after the **ADD** instruction, the ALU output reflects the arithmetic result, and the Accumulator is updated accordingly on the enabled clock edge.

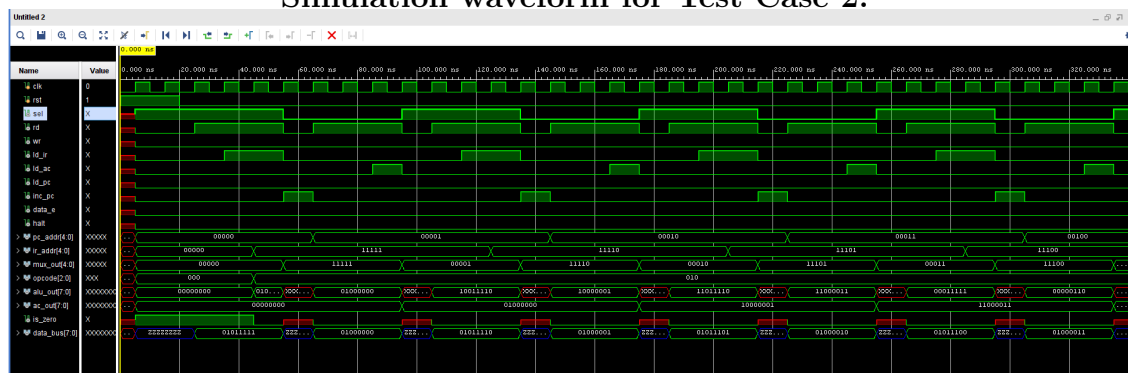
The later store operations exercise the bidirectional data bus because the processor must drive Accumulator data back into memory only when `wr` and `data_e` are asserted. This test therefore demonstrates correct cooperation between the ALU, Accumulator, Memory, Program Counter, and Controller for a normal sequential program without branching.

V.2.2 Test Case 2: Endless Execution Without HLT (input2.txt)

This test verifies the processor behaviour when a program does not contain a HLT instruction.

All 32 memory locations are filled with ADD instructions whose operands descend from address 31 to address 0. After the Program Counter reaches the final address, it wraps around and continues fetching from the beginning, causing the processor to execute an infinite loop instead of stopping.

Simulation waveform for Test Case 2.



The waveform confirms that `halt` is never asserted, while the fetch, read, Accumulator update, and Program Counter increment signals continue repeating. This demonstrates that the processor only stops when a valid HLT instruction is decoded; otherwise, execution proceeds continuously through memory.

V.3 Performance and Synthesis Evaluation

The processor is evaluated using the timing assumptions from the simulation testbench. The clock toggles every 5 ns, producing a 10 ns clock period and an equivalent operating frequency of 100 MHz. Most instructions complete the full eight-state controller sequence, while the HLT instruction stops earlier at the `OP_ADDR` state.

V.3.1 Instruction Latency

Instruction latency is calculated as:

$$\text{Latency} = \text{CPI} \times T_{clk}$$

For normal instructions such as ADD, AND, XOR, LDA, STO, JMP, and SKZ, the CPI is 8 cycles. Therefore, their latency is:

$$8 \times 10 \text{ ns} = 80 \text{ ns}$$

For HLT, the processor halts after 5 cycles, so its latency is:

$$5 \times 10 \text{ ns} = 50 \text{ ns}$$

Thus, the average instruction latency is approximately 78 ns when normal instructions dominate the program and HLT appears only at the end.

V.3.2 Throughput

The instruction throughput can be estimated from the clock frequency and average CPI:

$$\text{Throughput} = \frac{f}{\text{CPI}_{avg}}$$

Using the dominant CPI value of 8 cycles per instruction:

$$\text{Throughput} = \frac{100 \text{ MHz}}{8} = 12.5 \text{ MIPS}$$

This means the processor can complete approximately 12.5 million standard instructions per second under the assumed 100 MHz clock.

V.3.3 Program Execution Time

Program execution time is calculated as:

$$T_{exec} = \text{Total cycles} \times T_{clk}$$

For `input1.txt`, the executed program contains six normal instructions followed by one HLT instruction. Since each normal instruction requires 8 cycles and HLT requires 5 cycles, the total cycle count is:

$$6 \times 8 + 5 = 53 \text{ cycles}$$

Therefore, the execution time of Test Case 1 is:

$$T_{exec} = 53 \times 10 \text{ ns} = 530 \text{ ns}$$

For `input2.txt`, there is no final HLT instruction, so the program does not have a finite total execution time. One complete pass through the 32 memory addresses contains 32 normal ADD instructions, requiring:

$$32 \times 8 = 256 \text{ cycles}$$

The time for one complete loop is therefore:

$$T_{loop} = 256 \times 10 \text{ ns} = 2560 \text{ ns} = 2.56 \mu\text{s}$$

After each loop, the Program Counter wraps around and execution continues, so the total runtime of Test Case 2 is theoretically infinite until reset or external interruption.

VI Conclusions

This project successfully designed and verified a simple 8-bit RISC processor using Verilog HDL at the RTL level. The processor follows a modular structure consisting of seven main blocks: Program Counter, Address MUX, Memory, Instruction Register, Accumulator Register, ALU, and Controller. Its accumulator-based datapath and shared memory architecture allow it to execute the required instruction set, including arithmetic, logic, memory, control-flow, and halt operations. Unit-level simulations confirmed the correct behaviour of individual modules, while system-level tests demonstrated complete program execution. The eight-state FSM controller provides a clear and deterministic instruction cycle, making the processor easy to analyse and debug. Although the design is non-pipelined and therefore executes each instruction over multiple clock cycles, this simplicity is suitable for the educational purpose of the project. Overall, the project strengthened the team's understanding of RTL modelling, datapath control, FSM design, and processor verification.